

30221/39521- Sistemas Distribuidos Universidad de Zaragoza,
curso 2025/2026

Práctica 1: Conceptos y mecanismos básicos



**Escuela de
Ingeniería y Arquitectura**
Universidad Zaragoza

Rubén Agustín Galdeano (820829) \

Antonio González Almela (143045)

ÍNDICE

EJERCICIO 1.....	3
INTRODUCCIÓN.....	3
ANÁLISIS RED.....	3
EJERCICIO 2.....	5
INTRODUCCIÓN.....	5
DISEÑO E IMPLEMENTACIÓN.....	5
EJERCICIO 3.....	8
INTRODUCCIÓN.....	8
DISEÑO E IMPLEMENTACIÓN.....	8
SERVIDOR SECUENCIAL.....	8
SERVIDOR CONCURRENTE.....	8
SERVIDOR CON “THREAD POOL”	8
MASTER-WORKER.....	8

EJERCICIO 1.

INTRODUCCIÓN

En este ejercicio se analizan las prestaciones de los protocolos **TCP** y **UDP**, midiendo los tiempos que tardan en establecer conexión y transmitir datos en distintos escenarios: cuando el servidor está o no disponible, y cuando cliente y servidor se encuentran en la misma máquina o en máquinas diferentes.

ANÁLISIS RED

TCP es un protocolo de comunicación orientado a conexión, ya que ofrece una transmisión fiable y ordenada. Cuando el otro extremo no está activo y se intenta establecer la conexión, el **timeout** lo gestiona el sistema operativo, lo que nos da el **TIME1**. En local, el coste es menor porque el loopback está dentro de la misma máquina y el retardo es mínimo, aunque la diferencia no resulta muy grande.

Cuando el otro extremo sí está activo, el **TIME2** corresponde al coste del *handshake* de tres pasos: envío del **SYN**, recepción del **SYN+ACK** y envío del **ACK**. Aquí el tiempo aumenta, pero tampoco se nota mucha diferencia entre localhost y otra máquina, ya que las Raspberry Pi están en el mismo switch y apenas existe retardo de propagación como el que habría en redes más grandes con routers y varios switches.

Por otro lado, **UDP** es un protocolo no orientado a conexión, sin garantías de fiabilidad ni de orden. Si el otro extremo no responde con un datagrama, no es posible saber si el mensaje llegó, lo que refuerza la idea de que es un protocolo más “ligero” pero menos seguro. En este caso, el **TIME1** no se puede medir porque no hay fallo de conexión que detectar. Con el **TIME2** sí podemos obtener un tiempo de ida y vuelta, y se observa que es menor que en TCP, ya que UDP tiene menos sobrecarga de control. Al igual que en el caso de TCP, no hay una diferencia grande entre localhost y otra máquina porque todas las Raspberry Pi están conectadas al mismo switch.

SRC	DST	PROTO	TIME1*	TIME2**
192.168.3.13	192.168.3.14:29281	TCP	720.656 μ s	1.294168 ms
192.168.3.13	127.0.0.1:29281	TCP	549.733 μ s	1.142021 ms
192.168.3.13	192.168.3.14:29281	UDP	-	735.194 μ s
192.168.3.13	127.0.0.1:29281	UDP	-	743.564 μ s

*TIME1 es el tiempo desde que se manda el segmento SYN el TCP o el primer datagrama en UDP, hasta que falla la conexión.

**TIME2 es el tiempo desde que se manda el segmento SYN el TCP o el primer datagrama en UDP, hasta que contesta el otro extremo.

EJERCICIO 2.

INTRODUCCIÓN

El objetivo de este ejercicio es diseñar e implementar un algoritmo básico de barrera distribuida en **Go**, utilizando como base el código de apoyo proporcionado (barrier.go). La idea central es que cada proceso alcance la barrera, notifique su llegada, y no pueda continuar hasta que todos los procesos involucrados hayan hecho lo mismo. Esto garantiza un control coordinado del avance de la aplicación y evita que algún proceso se adelante sin que los demás estén sincronizados.

Durante las pruebas del algoritmo se observó que lo anterior no era suficiente. Puede ocurrir que un proceso reciba todos los mensajes de confirmación de los demás y aún así no haya enviado sus propios mensajes a todos los demás procesos, sobre todo si estos se acaban de levantar. Para evitar este escenario, se añadió una condición adicional: cada proceso debe **haber enviado sus mensajes de confirmación y haber recibido los mensajes de confirmación de los demás** antes de poder continuar. De esta manera, se asegura que la barrera distribuida cumpla correctamente su función y que todos los procesos están completamente sincronizados antes de avanzar.

DISEÑO E IMPLEMENTACIÓN

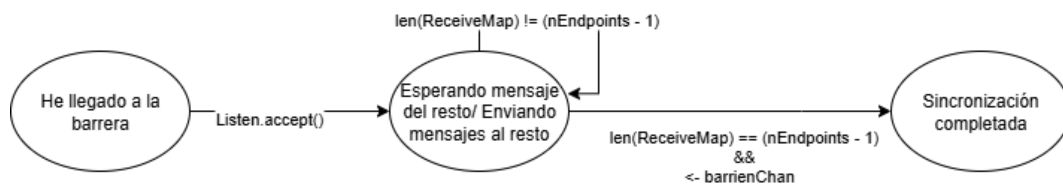
Para este ejercicio se han llevado a cabo dos diseños, la diferencia base entre los dos procesos se basa en cuál es la condición para salir de la barrera y poder decir que ya se ha realizado la sincronización.

En el primer diseño la condición de salida es que el resto de procesos te hayan enviado el mensaje de que ya están en la barrera. Esto es insuficiente porque debido a problemas de la red puede ser que un proceso aún no haya enviado sus dos mensajes de que ya estaban en la barrera. Provocando esto bloqueos en uno o más de los procesos de la barrera y no consiguiendo la sincronización requerida.

Esto se ha solucionado mediante el segundo diseño, el cual añade una condición más para poder decir que la sincronización ha sido realizada correctamente. La condición es que no solo se hayan recibido los mensajes del resto de extremos, si no que además se ha de haber enviado todos los mensajes y haber sido recibidos por el resto de endpoints. Para ello nos apoyamos en TCP que nos proporciona una conexión, fiable y ordenada.

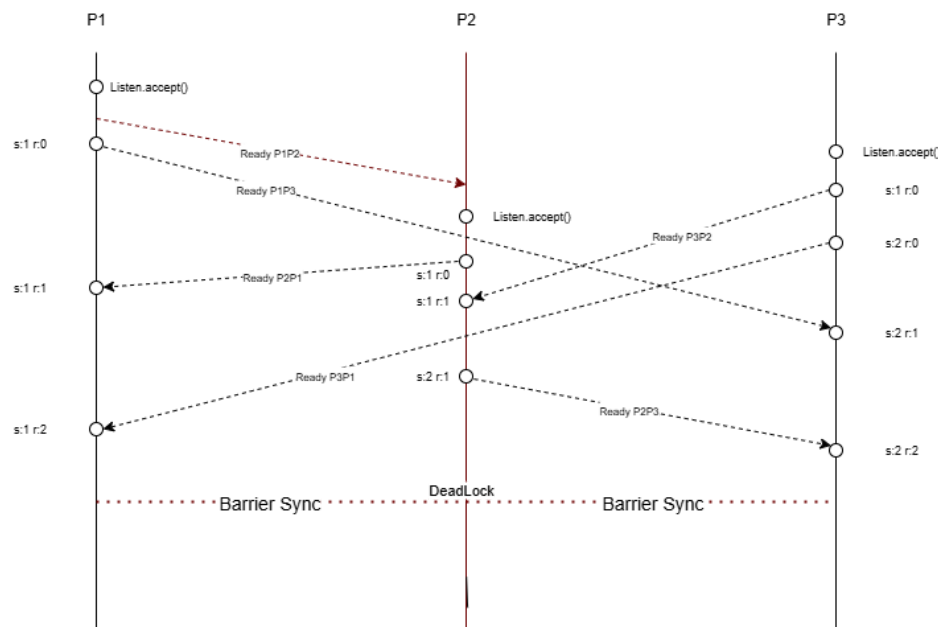
Esta máquina de estados refleja, el algoritmo de la sincronización de barrera, se puede apreciar que la condición para tener la barrera sincronizada es haber recibido todos los mensajes de estoy en barrera del resto de endpoints y que en el canal de “**barrierChan**” se haya escrito para sincronizar cuando se puede continuar con el proceso principal.

Esto causa problemas de bloqueo entre los procesos si alguno de los procesos no ha conseguido entregar correctamente sus mensajes pero si ha recibido todos los mensajes del resto de extremos.



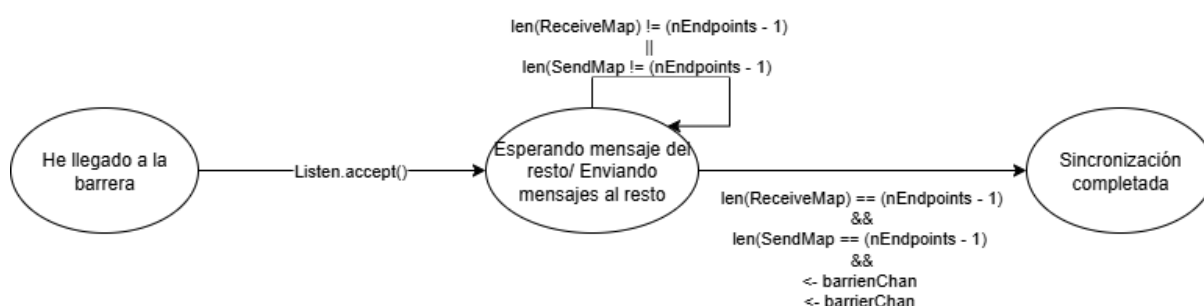
Este caso de bloqueo se ve mejor reflejado en el siguiente diagrama de secuencia, donde se puede ir siguiendo los diferentes envíos y recepciones de mensajes, manejados por cada proceso como 's' de sent y 'r' de receive respectivamente.

Se puede apreciar que el proceso P1 envía un mensaje “ReadyP1P2” antes de que P2 pueda aceptar conexiones por lo que no es capaz de entregar dicho mensaje, así que tendrá que reintentarlo. Antes de poder volver a mandar el mensaje, P1 ya ha recibido los dos mensajes necesarios, por lo que asume que la sincronización ya se ha completado sin tener en cuenta que P2 no ha recibido todavía su mensaje. Por consiguiente se provoca un “**Dead lock**” en el proceso P2 que nunca recibiera el mensaje que espera de P1.

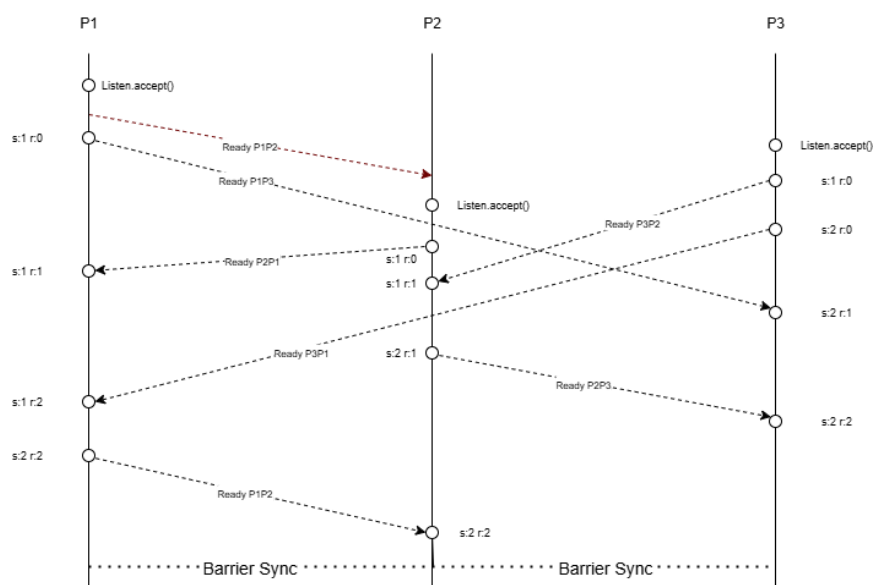


Para solventar la casuística comentada en el caso anterior, se propone el siguiente diagrama de estados donde el cambio más significativo es que ahora la condición para estar en “Sincronización completada” es que se hayan enviado todos los mensajes a los extremos y se hayan recibido todos los mensajes del resto de extremos. Para poder confirmar que se ha enviado todos los mensajes y que el otro extremo los ha recibido correctamente, nos apoyamos en la fiabilidad que aporta TCP, asumiendo que si enviamos el segmento con el mensaje y recibimos el ACK correspondiente, el mensaje ha sido entregado.

Debido a esta nueva condición, se debe leer dos veces del canal “barrierChan”, una para la condición de haber recibido todos los mensajes y otra para la condición de haber enviado todos los mensajes.



En este diagrama de secuencia se muestra cómo en contraposición del diagrama del caso anterior, si se tiene en cuenta el envío de mensajes, las sincronización se lleva a cabo exitosamente. Ya que P1 vuelve a enviar su mensaje "ReadyP1p2", evitando los bloqueos.



EJERCICIO 3.

INTRODUCCIÓN

El objetivo de este ejercicio es conocer y comparar cuatro arquitecturas cliente-servidor, poniendo especial atención en el uso de un servidor con una bolsa de hilos de ejecución (*thread pool*) y en la arquitectura maestro-esclavo (*master-worker*).

Durante el desarrollo, se profundizó en la sincronización mediante el paso de mensajes, técnica que se utilizó tanto para coordinar las gorutinas en el tratamiento de peticiones y respuestas, como para gestionar correctamente el cierre del servidor.

Otro aspecto explorado fue el uso del paquete **gob**, aprovechando las funciones **Encode()** y **Decode()** para la serialización de datos. Gracias a este mecanismo, se habilitó el envío de mensajes en red con un formato portable, evitando incompatibilidades entre distintas arquitecturas de procesador y/o diferentes órdenes de bytes.

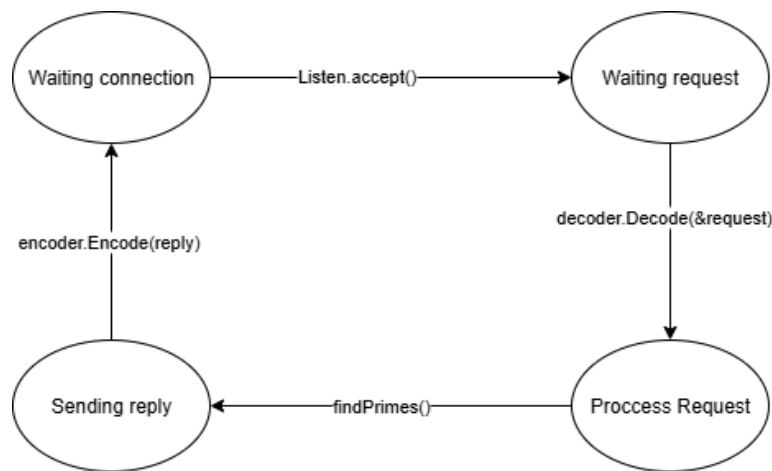
Las arquitecturas se han implementado sobre un servidor que calcula los números primos en un intervalo de números [a,b], y se ha desplegado el sistema sobre las Raspberry PI del laboratorio con comando ssh para poner en marcha el sistema. Dependiendo de la arquitectura a probar se han utilizado un número distinto de nodos.

DISEÑO E IMPLEMENTACIÓN

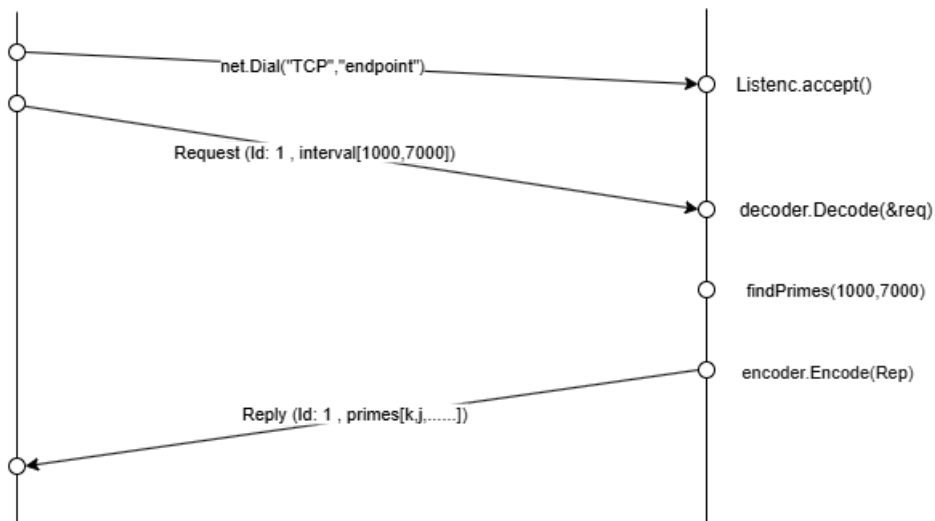
SERVIDOR SECUENCIAL

El esquema de esta arquitectura se basa en gestionar las peticiones siguiendo su orden de llegada. Cada petición se recibe, se procesa y, a continuación, se calcula el conjunto de números primos dentro del intervalo solicitado, enviando finalmente la respuesta al cliente

Tal y como se muestra en el siguiente gráfico, el servidor comienza en estado de espera hasta que recibe la conexión de un cliente. Una vez establecida, se realiza la operación **Decode()**, quedando bloqueado en la llamada **recv** del socket hasta recibir los datos. Después, se ejecuta la función **findPrimes()** para calcular los números primos, y el resultado obtenido se envía de vuelta al cliente mediante **Encode()**.



En el diagrama de secuencia siguiente se muestra la interacción entre un cliente y el servidor. El servidor está bloqueado en el `accept` y cuando el cliente hace el `connect` acepta la solicitud, haciendo el three-way-handshake. Posteriormente, el cliente debe enviar su petición con un identificador de petición y un intervalo de tiempo de números en los que se quiere encontrar los primos. El servidor decodifica la petición con `Decode` y calcula los primos con `findPrimes()`, después devuelve una respuesta con el mismo identificador que la petición y un array con todos los primos que se han encontrado en el intervalo.



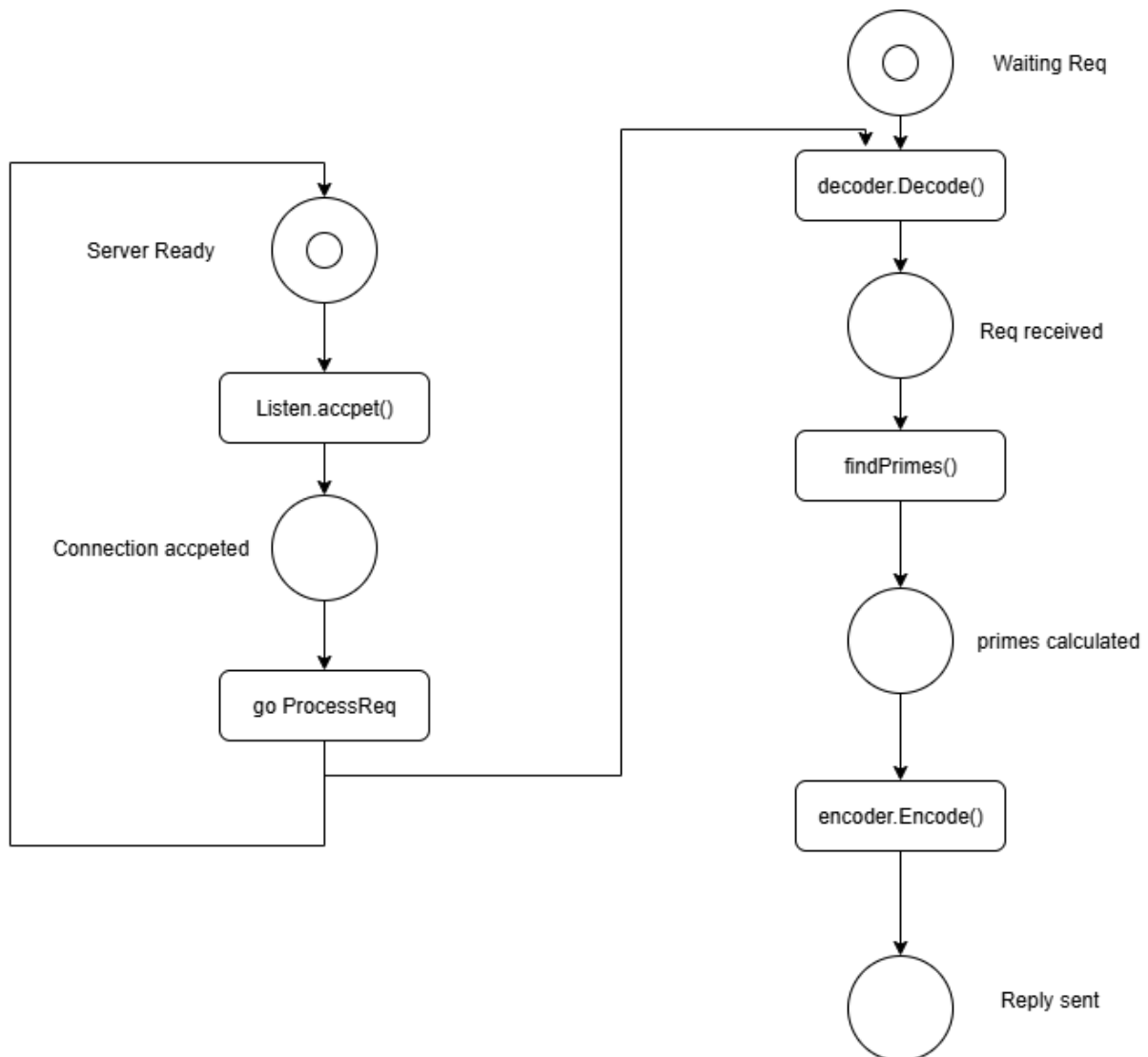
SERVIDOR CONCURRENTE

La arquitectura concurrente se trata de una extensión de la arquitectura anterior en la que solo podía haber un cliente calculando los números primos de su intervalo, con este diseño se consigue poder atender a múltiples clientes concurrentes, hasta que el procesador ya no tenga más núcleos y tenga que hacer pseudo concurrencia, ejecutando varias peticiones en un mismo core realizando cambios de contexto.

Además este diseño habilita la posibilidad de aceptar una nueva conexión entrante mientras se está procesando una petición de otro cliente. Otra ventaja respecto al esquema anterior es que si un cliente realiza una conexión y tarda en enviar su solicitud no bloquea todo el servidor, si no que este puede seguir atendiendo clientes.

Como se puede ver en la red de petri, la cual es un ejemplo de el hilo principal de ejecución y un hilo de gestión de un cliente, el hilo principal es el encargado de aceptar nuevas peticiones y lanzar un nuevo hilo con la conexión abierta con el nuevo cliente. El hilo principal volverá a quedarse preparado para una nueva conexión, mientras que el hilo creado se quedará esperando la petición del cliente que realizó la conexión y una vez recibida la petición realiza el cálculo y contesta al cliente con el resultado.

En cuanto el diagrama de secuencia, el esquema es el mismo que con el servidor secuencial solo que pudiendo recibir varias peticiones simultáneas. Esto provoca que si dos peticiones llegarán, en un t_1 y t_2 respectivamente, donde la diferencia entre los tiempos fuera lo suficientemente pequeña, podría darse el caso de que la respuesta no fuera ordenada con la petición ya que el planificador de los hilos podría decidir darle más tiempo de CPU al segundo hilo que recibió la petición en t_2 ,



SERVIDOR CON “THREAD POOL”

Este diseño es una variación del anterior, ya que pueden existir varios clientes concurrentes cuyas peticiones se procesan de forma simultánea. Al igual que antes, no se producen bloqueos durante la aceptación de nuevas conexiones ni en la espera de envíos de peticiones por parte de los clientes. La diferencia principal radica en que, en el esquema anterior, se podía crear una cantidad ilimitada de hilos de cálculo, lo cual resultaba intensivo en CPU y podía bloquear la máquina por insuficiencia de recursos. Para evitar este problema, en este nuevo diseño se utiliza un número fijo de hilos para el cálculo de los primos, al que llamamos *pool*.

Tal y como se observa en la red de Petri, este cambio introduce problemas de sincronización entre los hilos. Para afrontarlos, se ha empleado la técnica de paso de mensajes como mecanismo de gestión de la concurrencia.

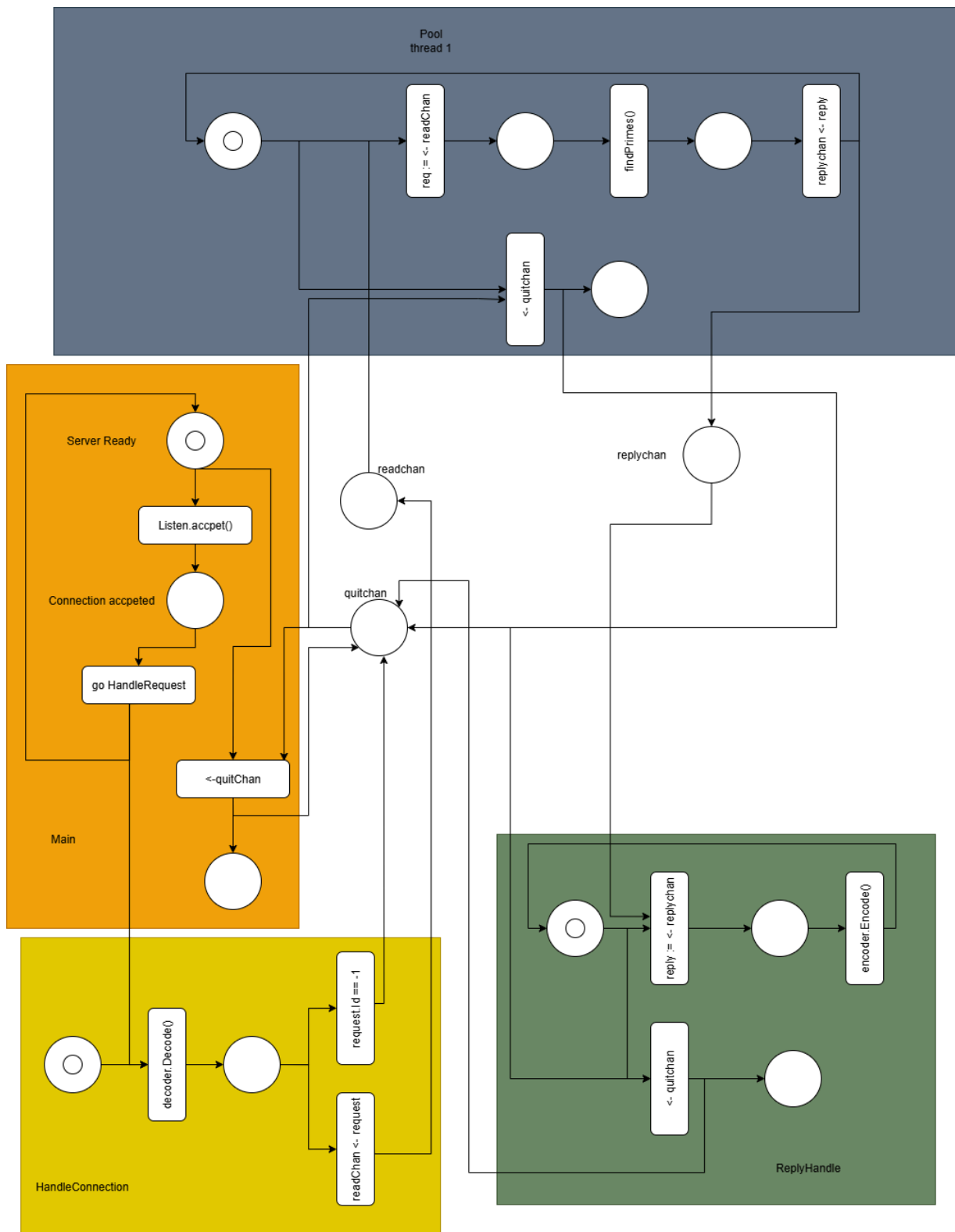
En este diseño se distinguen cuatro tipos de hilos de ejecución:

- **El main**, que en primer lugar inicializa la *pool* e inicia un hilo de respuestas a los clientes. Posteriormente, entra en un bucle en el que acepta nuevas conexiones y lanza un hilo específico para tratarlas, con el fin de estar siempre disponible para aceptar más clientes lo antes posible.
- **El hilo de tratamiento de conexiones entrantes**, que espera la petición de cada cliente y la introduce en un canal de peticiones para que los hilos de la *pool* las procesen. Además, registra la nueva petición en un mapa que asocia el identificador de la petición con la conexión del cliente, de manera que el hilo de respuestas pueda localizar a qué cliente debe enviar la contestación.
- **El hilo de respuestas**, que lee del canal las respuestas calculadas por la *pool*, localiza la conexión del cliente correspondiente en el mapa petición-conexión y envía la respuesta.
- **El hilo de procesamiento**, que representa a cada uno de los hilos de la *pool*. Estos leen del canal de peticiones: si no hay peticiones, permanecen bloqueados esperando; si hay varias, se sincronizan entre ellos para extraer una cada uno. A continuación, calculan el intervalo de primos y escriben la respuesta en el canal de salida, para que posteriormente se envíe al cliente.

Todos los hilos comparten un canal común de fin de trabajo llamado **quitChan**. Cuando un hilo de gestión de conexión detecta una petición con identificador **-1**, cierra dicho canal para notificar el final del trabajo y detiene el *listener*. Es importante que el *main* sea el último en

cerrarse para garantizar que el servidor finalice correctamente. Para ello, se ha implementado un mecanismo de sincronización basado en un contador del número de hilos activos: el hilo principal no termina hasta que este contador llega a cero, momento en el que se le comunica mediante un canal que puede finalizar.

Como ocurría con el diseño anterior, el diagrama de secuencia sería igual que del servidor secuencial pero con varios clientes pudiendo pedir conexiones y peticiones. Además de tener el mismo matiz temporal que el diseño del servidor concurrente anterior, por lo que dos peticiones consecutivas en el tiempo, cercanas, no tienen porque ser respondidas en el mismo orden.



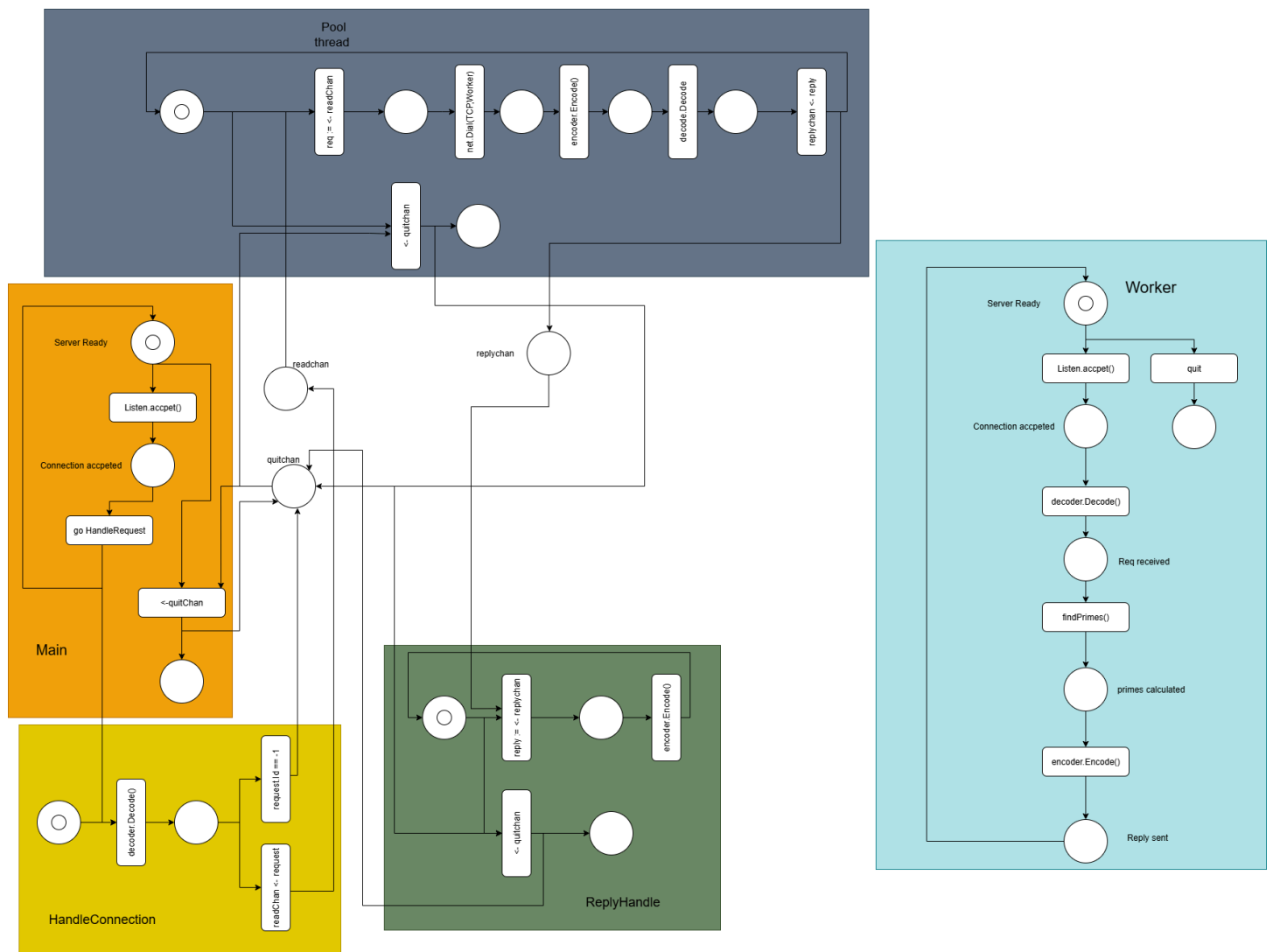
MASTER-WORKER

Este diseño puede considerarse una generalización del servidor concurrente con *pool*. Mientras que en el servidor con *pool* los hilos de procesamiento trabajan directamente dentro de la misma aplicación, en la arquitectura Master-Worker se introduce un maestro que reparte el trabajo a un conjunto de procesos trabajadores (*workers*). Estos *workers* pueden estar ejecutándose en la misma máquina, como procesos separados comunicados por red, o en diferentes máquinas distribuidas.

En nuestra implementación, esta arquitectura se ha desarrollado sobre TCP. Se mantienen los mismos tipos de hilos que en el servidor con *pool*: el *main*, el hilo de gestión de conexiones, el hilo de respuestas y el hilo de procesamiento. La diferencia principal se encuentra en este último: en lugar de calcular directamente el intervalo de primos, establece una conexión TCP con un *worker*, le envía la petición tal y como la recibió del canal de peticiones, y queda bloqueado esperando su respuesta. Cuando el *worker* responde, el hilo de procesamiento escribe el resultado en el canal de respuestas para que el hilo de envío al cliente pueda devolverlo.

El *worker* se comporta como un servidor secuencial: acepta conexiones, recibe la petición, realiza el cálculo y envía la respuesta, tras lo cual vuelve a quedar bloqueado en el *accept* esperando nuevas solicitudes. De esta forma, cada *worker* funciona de manera independiente y secuencial, pero el maestro puede tener tantos *workers* como hilos de procesamiento en la *pool*, lo que permite escalar el sistema tanto en procesos locales como en máquinas distintas.

En la red de Petri se aprecia que la estructura general es la misma que en el servidor con *pool*, pero con la incorporación del proceso *worker*, representado como un servidor externo que solo se comunica por red. Es importante destacar que: Los canales (**reqChan**, **replyChan**, **quitChan**) continúan siendo el mecanismo de sincronización central, y aparecen reflejados como puntos de paso de mensajes en el Petri.



En este caso, el diagrama de secuencia adquiere más relevancia que en los diseños anteriores, ya que la incorporación de los *workers* introduce un mayor número de mensajes por la red para coordinar los procesos.

Se puede observar cómo dos clientes envían peticiones al *master*. Este, al recibirlas, las reenvía a los *workers*, que las procesan de manera concurrente. En el ejemplo mostrado, la respuesta a la petición 2 llega antes que la respuesta a la petición 1. Esto ilustra la casuística ya comentada: las respuestas no tienen por qué respetar el orden de llegada de las peticiones. Sin embargo, cada respuesta se entrega correctamente al cliente que realizó la solicitud, manteniendo la coherencia en la comunicación.

Asimismo, se incluye un ejemplo de lo que ocurre cuando un cliente envía una petición de fin con **request.id = -1**. En este escenario cobra especial importancia que el *main* finalice después del resto de hilos, de forma que los *workers* puedan cerrarse correctamente. Para conseguirlo se ha seguido el mismo mecanismo que en el servidor con *pool*: una variable de conteo de hilos activos, modificada en exclusión mutua mediante funciones que actúan como monitor. Cuando el número de hilos llega a cero, el hilo principal —que se encontraba bloqueado esperando esta condición— recibe la señal que le permite terminar.

